

PES University — Dept. of CSE
Lab 3: Component Modelling & Architectural Pattern Selection

Applied to Problem Statement #49

Webhook Ingestion & Retry Mechanism Hub

Component-level architecture for the resilient webhook gateway from Labs 1 & 2: ingesting third-party events, logging payloads, forwarding to internal services, and managing dead-letter queues with exponential backoff retries.

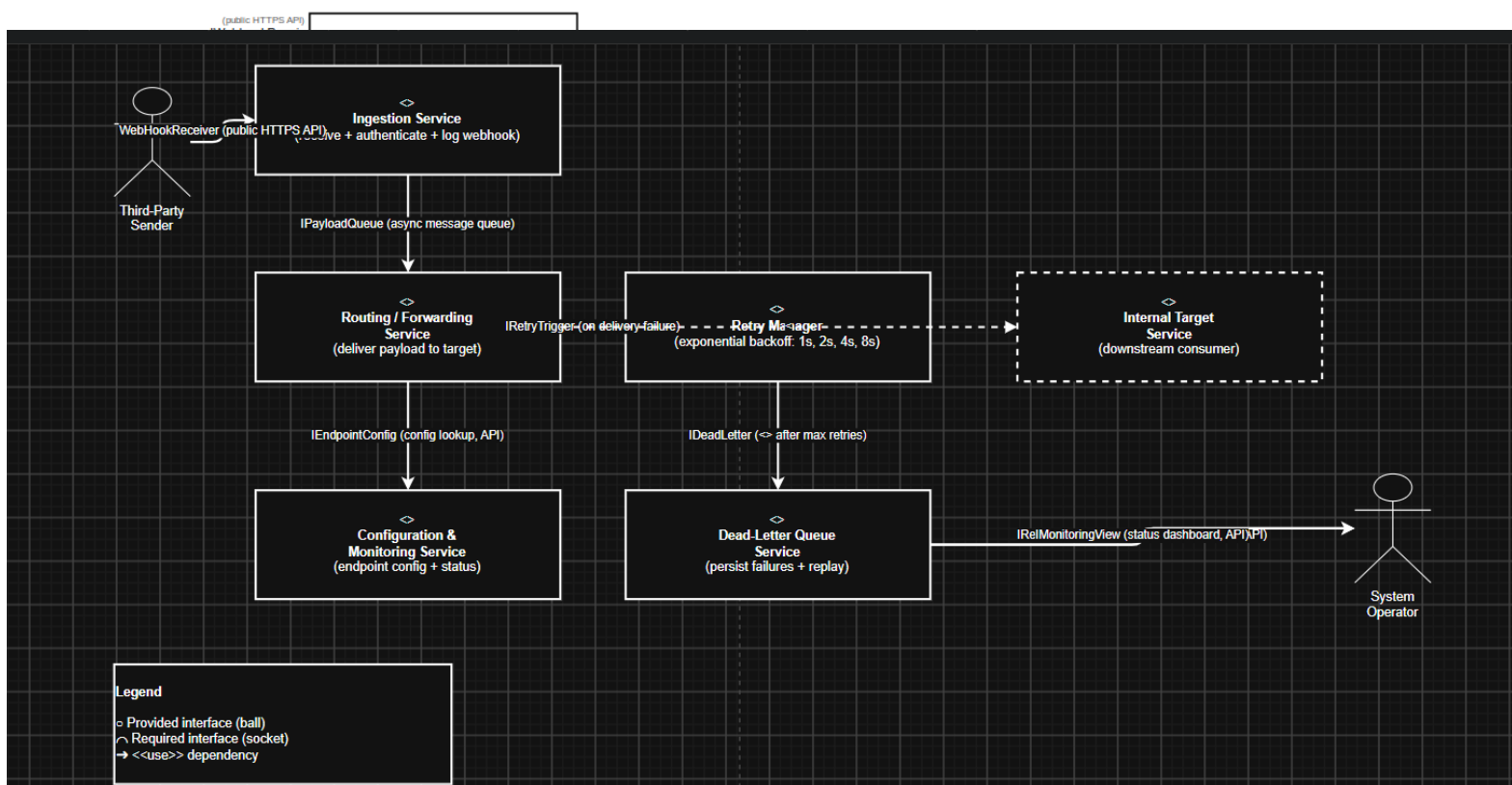
Contents

1. UML Component Diagram
2. Architectural Justification

1. UML Component Diagram

Five components: Ingestion Service, Routing/Forwarding Service, Retry Manager, Dead-Letter Queue Service, and Configuration & Monitoring Service, organized as independently deployable microservices. External actors are the Third-Party Sender, the Internal Target Service, and the System Operator. Interfaces shown include IWebhookReceiver, IPayloadQueue, IRetryTrigger, IDeadLetter, IEndpointConfig, IReplayControl, and IMonitoringView, using standard provided (ball) / required (socket) UML notation, plus a <<use>> dependency from Routing/Forwarding Service to the external Internal Target Service.

Webhook Ingestion & Retry Mechanism Hub — UML Component Diagram (Microservices Architecture)



2. Architectural Justification

Architecture Selection: We chose Microservices Architecture for the Webhook Ingestion & Retry Mechanism Hub.

Architectural Choice

The system is decomposed into independently deployable services: Ingestion Service, Routing/Forwarding Service, Retry Manager, Dead-Letter Queue Service, and Configuration & Monitoring Service. Services communicate through defined interfaces and an asynchronous message queue between ingestion and downstream processing, rather than through shared in-process calls.

Reason 1 — Independent Scaling Under Uneven Load

NFR-001 requires the ingestion endpoint to sustain up to 2,500 requests/second, while retry volume depends on unrelated factors such as a downstream target being temporarily degraded. These two workloads do not scale together: a burst of inbound webhooks does not imply a burst of retries, and vice versa. Microservices let the Ingestion Service and the Retry Manager scale independently based on their own load, which a single monolithic layered process could not do without over-provisioning the whole system for its busiest component.

Reason 2 — Fault Isolation Between Ingestion and Delivery

FR-001 and NFR-002 require that failures in delivering to a downstream target never cause dropped or lost webhooks, and that the gateway maintain 99.9% uptime. By separating Ingestion Service from Routing/Forwarding Service and Retry Manager behind a message queue, a struggling or completely unavailable Internal Target Service only slows the Retry Manager — it cannot block or crash the public-facing Ingestion Service, which keeps accepting and persisting new webhooks regardless.

Security Advantage

Only the Ingestion Service is exposed to the public internet and performs authentication of inbound requests (IWebhookReceiver). The Retry Manager, Dead-Letter Queue Service, and Configuration & Monitoring Service sit behind an internal network boundary and are reachable only from other services or the System Operator, not directly from third-party senders. This confines the attack surface to a single, hardened entry point instead of exposing every internal service to external traffic.

Performance Benefit

The asynchronous IPayloadQueue interface between Ingestion Service and Routing/Forwarding Service lets the public endpoint acknowledge the sender immediately after persisting the payload, without waiting for the downstream delivery (and any retries) to complete. This decoupling is what allows the Ingestion Service to sustain high throughput independent of how slow or unreliable a given Internal Target Service is at any moment.